```
Given a project named "Rails 5 Test Prescriptions"
Given a project named "Evil Master Plan"
```

In each case the step definition would create a project with the given name.

You can go even further in terms of putting data in the Cucumber feature file. Cucumber allows you to append a table of data to a step. It's a very clever feature, and like a lot of very clever features it should be used judiciously.

You use something like Markdown table syntax to create the table. In the feature file it might look something like this:

```
Given the following users:
    | login| email            | password| password_confirmation|
    | alpha| alpha@example.com| alpha1  | alpha1               |
    | beta | beta@example.com | beta12  | beta12               |
```

The step with the table needs to end with a colon. The table uses pipe characters to delimit entries. They don't have to line up, but usually you'll want them to.

When Cucumber matches a step definition to a step that has a table, the table becomes an argument to the step definition's block—if there are other regular-expression matches, the table is the last argument. The argument is a special Cucumber data type, and there are a few different ways you can deal with it. Most commonly you'll deal with it as an array of hashes, where the keys are the first row of the table and every subsequent row provides a set of values, like so:

```
Given /^the following users$/ do |user_data|
  User.create!(user_data.hashes)
end
```

You can do something similar with a large string literal:

```
Given I have typed the following
  """
  some big amount of text
  """
```

That's an indented line, three quotation marks, some text, and then three more quotation marks at the end. The text inside the triple quotes will be passed as the last argument to the step definition's block.

If you really want to have fun, you can combine scenarios and tables to create a loop called a *scenario outline*, like so:

```
Scenario Outline: Users get created
  Given I go to the login page
  When I type <login> in the login field
```